4COSC001W: Lecture Week 2

1. Part A
    - Built-in Functions - int( ), float( ),
    - Introduction to User Defined Functions
    - Keyboard input
    - Pseudocode
    - Augmented Assignment Operators
2. Part B
    - Debugging, Program Errors, Try and Except

Built-in Functions – print( )

- A function is a piece of code written to carry out a specified task.
- We have seen the **print() function** is used to send content to the screen
- Python is case sensitive. Use print(), rather than Print( ) or PRINT( ).

```
print()                 # empty line
print('Hello')          # Hello
print(42)               # 42

greeting = 'Hello'      # Assign a string to variable
print(greeting)         # print variable value
```

User Defined Functions

- We can also create our own functions

```
def function_name(parameter1, parameter2, ... parameterN):
      function_block
```

• A Python function is defined using the `def` **keyword**, followed by the function **name**, followed by **parentheses** ( ) ending with a **colon**.
• Parameters are optional. The parentheses ( ) may contain a list of parameters (as shown above) or be empty (as below).

```
def function_name():
      function_block
```

• The code to be executed by the function is indented

Calling User Defined Functions

- Code blocks inside a user defined function are not executed automatically.
- To execute the function we call it by it's name and if we have any parameters listed in the function definition we send values to the function in the same order

```
function_name(parameter1, parameter2, ... parameterN):
```

When we send a specific value for a parameter into a function, this is called an argument.

Calling User Defined Functions

Example: a user defined function that calculates the volume of a box

1. Define the function

```
def calcBoxVol (length, width, height):
        vol=length*width*height
        print (" The volume is ",vol)
```

Note- all the code executed within the function is indented

Now to execute the function we call it by name and include the arguments we want to assign to the three parameters

```
calcBoxVol(10,2,5)
```

So this call to calcBoxVol sends 10 to the length value 2 to the width and 5 to the height.

Calling User Defined Functions

- Functions are powerful tools because they are re-usable
- We can call them as many times as we like and send different arguments into the function to get a different result.
- We can send variable values into a function if we need to

Example: Let's call our calcBoxVol function again with a different box size

```
new_length=15
new_width=20
new_height=12

calcBoxVol(new_length, new_width, new_height)
```

So this call to calcBoxVol sends 10 to the length value 20 to the width and 12 to the height.

As long as the arguments we send into the function are the correct data type the function will process the data

Keyboard input - input( ) function

- Typical flow of a simple computer program
  1. Input data,
  2. Work with data,
  3. Output answer/s.
- **input( ) function -** program stops and waits for the user input. When the user presses Return or Enter, the input is returned to the program **as a string**.

```
text = input()
```

- **input( )** can take text as an argument (e.g., a prompt stating what input is expected):

```
name = input('What...is your name?\n')
```

Keyboard input - input( ) function

- Note: \n represents a **newline** (line break) so that any user input will appear below the prompt message.
- If you expect the user to type an integer, convert the value to integer.
- int( ) returns an integer so you need to save the result into a variable.

```
prompt = 'What is 38 plus 5?\n'
answer = input(prompt)
answer = int(answer)
```

Conversion functions – int( ), float( ), str( )

- int( ) function takes any value and converts it, if it can, to an integer:

```
a = int('32')        # a is integer 32
b = int('Hello')     # produces an error
```

- int( ) function can convert floating-point values to integers, but it doesn't round off; it chops off the fraction part:

```
c = int(3.99999)     # c _____________________
d = int(-2.3)        # d _____________________
```

Conversion functions – int( ), float( ), str( )

- float( ) function converts integers and strings to floating-point numbers:

```
a = float(32)                    # 32.0
b = float('3.14159')             # 3.14159
```

- str( ) function converts its argument to a string (seen earlier):

```
c = str(32)                      # '32'
d = str(3.14159)                 # '3.14159'
```

Pseudocode (1)

- To use pseudocode, all you do is write what you want your program to do in short English phrases.
- Allows you to focus on the algorithm logic without being distracted by details of language syntax.
- Pseudocode is not a rigorous notation, since it is read by people. However, for consistency you can use a particular style.

Pseudocode (2)

- SEQUENCE – linear, one task performed sequentially after another.

- Common Action Keywords for input, output and processing:
    - Input: INPUT, READ, GET
    - Output: PRINT, DISPLAY, OUTPUT
    - Compute: COMPUTE, CALCULATE
    - Initialize: SET
    - Add one: INCREMENT

Pseudocode - example 1

- Example 1: Calculate the average of three numbers. Example pseudocode:

```
INPUT num_1
INPUT num_2
INPUT num_3
average <- (num_1 + num_2 + num_3) / 3
PRINT average
```

- Pseudocode is a set of instructions to solve a programming problem – carried out before the implementation stage.
- The implementation stage involves creating the program in chosen programming language **using the pseudocode for guidance.**

Pseudocode – example 2 & 3

- Example 2: Write the program to store 5 into *total*. Then add 1 onto *total* and store the answer back into *total*.

Pseudocode:
```
total <- 5
total <- total + 1
```

- Example 3: Write the program to store 5 into cost, then add 3 onto cost and store the answer back in cost.
- Pseudocode :
```
cost <- 5
cost <- cost + 3
```

Practice Quiz Question

Write **pseudocode** to put zero into variable running_total. Then write separate instructions to add the following numbers onto what is in the variable, adding one number at a time 5, 8, 2, 3. Print running_total.

Practice Quiz Question

1. Write a program to get and print the number of pets a user has.

Practice Quiz Question (Solution for Tutorial)

2. Write the Python program using the following pseudocode.

```
INPUT num_1
INPUT num_2
total <- num_1 + num_2
PRINT total
```

Practice Quiz Question (Solution for Tutorial)

3. Write the Python program using the following pseudocode.

```
INPUT cost_of_item
INPUT cash_paid (e.g., 10 for £10)
CALCULATE change
PRINT change
```

Quiz Question (Solution for Tutorial)

4. Write the program to calculate the average of 3 numbers.
Pseudocode:

```
INPUT num_1
INPUT num_2
INPUT num_3
average <- (num_1 + num_2 + num_3) / 3
PRINT average
```

Augmented Assignment Operators

- Example Python code: **count = count + 1**
- Python does not support ++ and -- operators to add/minus 1 (used in other programming languages). However, the code can be shortened: **count += 1**

- Complete the examples:

Operator	Example	Is equivalent to
+=	x += 3	x = x + 3
/=		x = x / 5
*=		x = x * 4
-=		x = x – 2

Lecture 2 -

Part A - covered

- Built-in Functions - `int(), float(), str(), input()`
- Pseudocode
- Augmented Assignment Operators . E.g., `x += 3`

Intermission break – 5 mins

Part B

- Debugging
- Program Errors: syntax errors, runtime errors, and semantic errors.
- Common error messages
- Try …. Except - Python Exception Handling

Indentation

- Over the next few weeks we create code ‘blocks’ for conditions and loops.
- Indentation refers to the spaces at the beginning of a code line.
- Where in other programming languages the indentation in code is for readability only, the indentation in Python is very important.
- Python uses indentation to indicate a block of code. (Note: many other programming languages indicate blocks with curly braces {}).

- An indentation example will be shown for the **try … except** concept

Debugging

- Bug - An error in a program
- Debugging - The process of finding and correcting bugs
- The ability to debug your programs is an important skill!
- Three kinds of errors can occur in a program:
    - syntax errors,
    - runtime errors, and
    - semantic errors.

Debugging - Syntax error

- Syntax error - Refers to the structure of a program and the rules about that structure.
  - Look for missing parentheses, quotation marks, or commas.
  - E.g., parentheses have to come in matching pairs) is a **syntax error**

- Python displays an error message and quits, and you will not be able to run the program.
- It is normal when you start programming to spend a lot of time tracking down syntax errors!

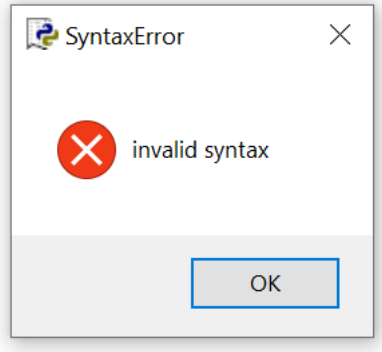
Debugging - SyntaxError

- **SyntaxError**: Example:

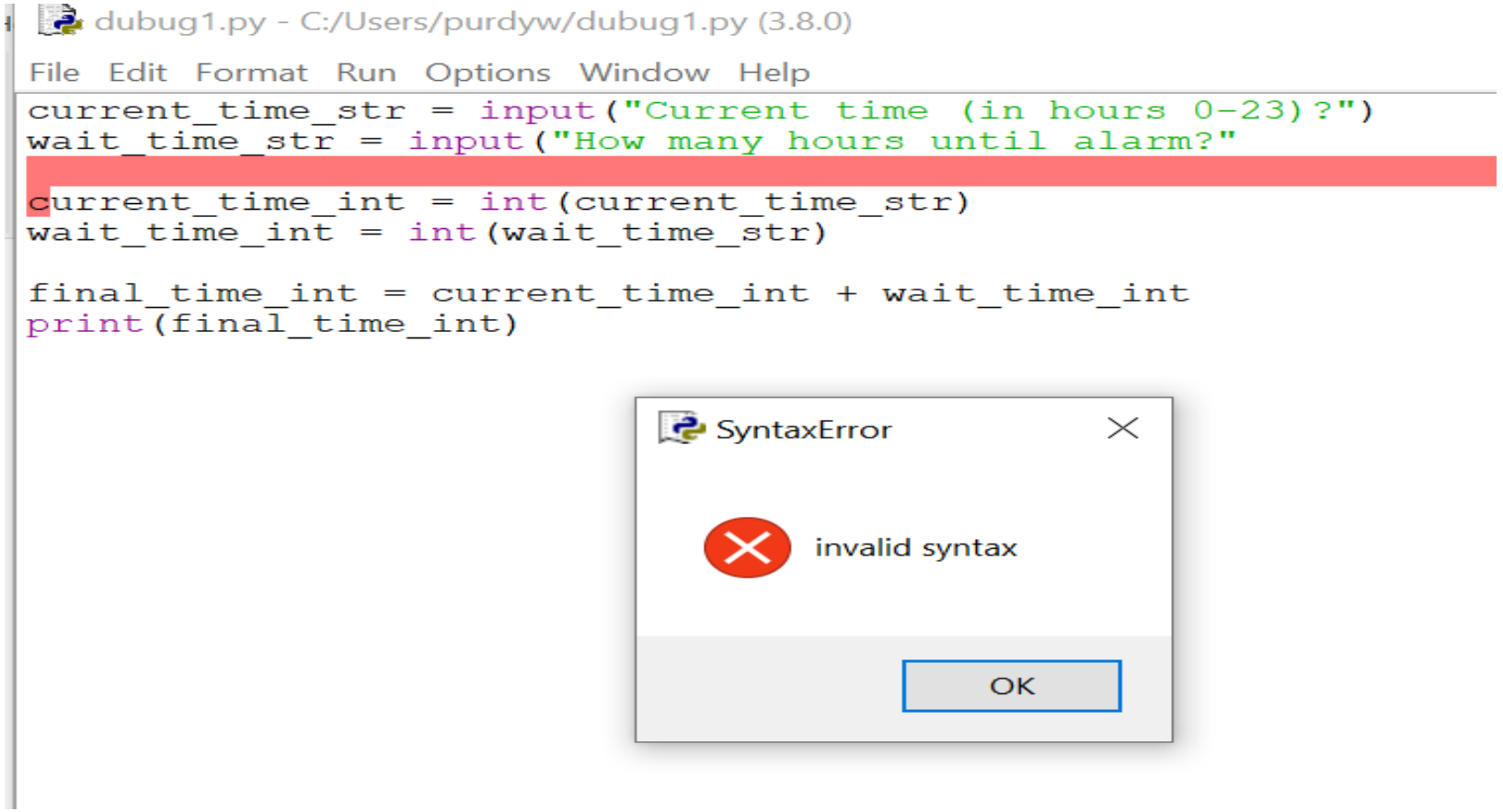
```
current_time_str = input("Current time (in hours 0-23)?")
wait_time_str = input("How many hours until alarm?"

current_time_int = int(current_time_str)
wait_time_int = int(wait_time_str)

final_time_int = current_time_int + wait_time_int
print(final_time_int)
```



To find: Look for missing parentheses, quotation marks, or commas. IDLE might highlight a line of code that does not have an error. So, work back line by line until you find the syntax error.


dubug1.py - C:/Users/purdyw/dubug1.py (3.8.0)
File Edit Format Run Options Window Help
current_time_str = input("Current time (in hours 0-23)?")
wait_time_str = input("How many hours until alarm?"
current_time_int = int(current_time_str)
wait_time_int = int(wait_time_str)
final_time_int = current_time_int + wait_time_int
print(final_time_int)
SyntaxError
invalid syntax
OK

Debugging – Runtime & Semantic Errors

Runtime error

- The error does not appear until after the program has started running. Also called **exceptions** (e.g., something exceptional has happened).
- We will look at some errors related to exceptions:
  - **NameError, TypeError, ValueError, ZeroDivisionError**

Semantic error

- Semantics – the meaning of a program.
- Semantic error - An error in a program that makes it do something other than what the programmer intended.
- The program will run without generating error messages, but it will not do the right thing.

Debugging

Debugging can involve finding clues.

- Error Messages - Understanding error messages can help you.
- Using print to debug your code - Using extra print( ) statements to display the value of your program's variables is a useful way to figure out what’s is really going on in your program
- Note that sometimes an error message is caused by something that has happened earlier in the program so you might need to work backwards (up) from the error.

Debugging - TypeError

- TypeError: often when an expression tries to combine two types that are not compatible (e.g., integer and string).

Example:

```
a = int(input('Enter number '))
x = input('Enter another number ')
total = a + x
```

Output:

```
Enter number 56
Enter another number 45
Traceback (most recent call last):
  File "C:/Users/purdyw/test2.py", line 3, in <module>
    total = a + x
TypeError: can only concatenate str (not "int") to str
>>>
```

Debugging – TypeError (continued)

- Previous Example:

```
a = int(input('Enter number '))
x = input('Enter another number ')
total = a + x
```

- **To find:** trace through variables to check they are the type you expected using **type().** Adding the following after line 2:

```
print(type(a))   # <class 'int'>
print(type(x))   # <class 'str'>
```

Debugging - NameError

- **NameError:** usually means you have used a variable before it has a value. Typo?
- Example: NameError: name 'wait_time_int' is not defined

```
current_time_str = input("Current time (in hours 0-23)?")
current_time_int = int(current_time_str)

wait_time_str = input("How many hours do you want to wait")
wait_time_int = int(wait_time_int)

final_time_int = current_time_int + wait_time_int
print(final_time_int)
```

- **To find:** Check the right hand side of assignment statements. You could also search for the exact word highlighted in the error message

Debugging - ValueError

- **ValueError**: Raised when an operation or function receives an argument that has the right type but an inappropriate value
- Example:
    - We ask the user for a number and instead they enter a character **t.**
    - We use int() to convert user input (string) to an integer but the value **t** cannot be converted to an integer number.
    - ValueError: invalid literal for int()

```
a = input('Enter number ')
a = int(a)
# do something here
# print result
```

- **To fix:** use try / except to catch this error.

Debugging - ZeroDivisionError

- **ZeroDivisionError:** with Python it is not possible to **divide** numbers
- by zero.
- If you attempt to **divide** by 0, **python** will throw a ZeroDivisionError

```
x = 100/0
```

- **To fix:**
  - Use an if statement to ensure that the value is not zero.
  - Or use try / except to catch this error (example later)

Exception Handling in Python

- What happens if a program is processing user input and the user enters the wrong type of data? The ‘try/except’ construct can prevent the program crashing with an error.
  - Exception: An error that occurs at runtime. Handle an exception by wrapping the block of code in a **try** . . . **except** construct.

Syntax:

```
try:
        Python commands
except:
        Exception handler
```

Exception Handling in Python – Example

```
a = "1"
try:
    b = a + 2
except:
    print(a, " is not a number")
```

- We try to add a number and a string (generates an exception). We trap the exception let the user know instead of letting the program crash.
- The above does not specify a specific exception (error) to handle, so can be used for **any** errors that occur while executing these commands.

Exception Handling - ValueError

- An exception for a specific exception (error). Syntax:

```
try:
        Python commands
except ExceptionType:
        Exception handle
```

- E.g., if input cannot be cast to an int it will generate a **ValueError**.

```
try:
    n = int(input("Please enter an integer: "))
except ValueError:
    print("Requires a valid integer!")
```

Exception Handling – ZeroDivisionError

- An exception for a specific exception (error). Syntax:

  try:

  Python commands

  except **ExceptionType:**

  Exception handle

- E.g., attempting to divide by zero will cause a **ZeroDivisionError**.

```
try:
    x = 2 / 0
except ZeroDivisionError:
    print("Cannot divide by zero!")
```

End Slide

- Part B covered:
  - Debugging
  - Program Errors:
    - syntax errors,
    - runtime errors, and
    - semantic errors.
  - Common error messages
  - Try and Except - Python Exception Handling